

StateLens: What Happens When a Model's Notepad Gets Full

DataForge 2026 · Pathway Track · StateLens team · github.com/yogeshroyal63-beep/Dataforge

The problem with remembering everything

Every time a standard Transformer generates the next token, it looks back at every single previous token. That comparison — query against every past key — is exact and reliable, but expensive: both the memory cost and the compute cost grow linearly with how long the conversation or document gets. [Vaswani et al., 2017]

Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.

For short inputs that's fine. For long documents, many in-context examples, or multi-step reasoning traces, the KV cache becomes the primary bottleneck. This is the design pressure that has motivated a wave of Post-Transformer architectures: Mamba, RetNet, GLA, and Pathway's Dragon Hatchling family.

Part A: the fixed-size notepad

Linear attention asks: what if we didn't keep every past key-value pair? Instead of appending each new token to a growing cache, we fold it additively into a single running matrix — a fixed-size 'notepad' — using the recurrence derived by Katharopoulos et al. (2020, ICML):

$$S_t = S_{t-1} + \phi(k_t) \cdot v_t, \quad y_t = \phi(q_t) S_t / \phi(q_t) z_t$$

Katharopoulos, A. et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML 2020. arXiv:2006.16236.

S_t is a fixed $d_k \times d_v$ matrix. Every new fact lands in the same matrix — memory footprint is $O(1)$ in sequence length, not $O(N)$. StateLens runs this exact recurrence live in the browser, in 16 dimensions, on a toy associative recall task.

"A fixed-shape recurrent state can process a sequence of unbounded duration without allocating a new memory slot for every token, but it can still forget through interference."

The StateLens Experiment page makes this falsifiable: drag the slider from $N = 2$ to $N = 20$ facts and watch the linear-attention model's recall accuracy fall while full attention stays near-perfect. The chart is not precomputed — the benchmark runs in the visitor's browser using `src/models/benchmark.js`.

Part B: BDH-CQ — the same trade-off at research scale

Pathway's BDH-CQ (Engdahl et al., 2026) is a Post-Transformer system that adapts to new tasks without updating its weights and without writing out a step-by-step reasoning text. It does this by accumulating demonstrations into a high-dimensional recurrent memory state.

The BDH-CQ technical report explicitly frames this memory mechanism using fast-weight and linear-attention language — the contextual memory is described as a state that accumulates additively per demonstration, analogous to the outer-product accumulation StateLens demonstrates in Part A. The parallel is not a metaphor imported from outside; it is the framing the paper itself uses.

At 150M parameters on ARC-AGI-1 (public evaluation set), BDH-CQ achieves 29.5% pass@2 at \$0.00070 per task (~0.85 s of H200 time). The authors report this as a Pareto-frontier result — more accuracy per dollar than comparable systems at the time of publication. This is a developer-reported benchmark result, not an independent evaluation. [Engdahl et al., 2026]

Engdahl, B. et al. (2026). BDH-CQ: In-Context Learning with Recurrent Latent Reasoning. arXiv:2608.09888.

The base BDH architecture (Kosowski et al., 2025) reformulates attention as Hebbian synaptic memory: neuron connections strengthen temporarily when co-active, implementing working memory without a separate cache. BDH-GPU, the hardware-efficient variant, uses ReLU-low-rank transformations with linear attention — making the connection to Part A's mechanism explicit at the architectural level. [Kosowski et al., 2025]

Kosowski, A. et al. (2025). The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507.

Part C: the failure that doesn't announce itself

At N = 14 facts, seed 5, query key 'Mango': full attention locates the correct value with high attention weight. Linear attention — with all 14 facts compressed into the same 16×10 matrix — returns a wrong answer with no error signal, no truncation, no visible warning. The model outputs a confident, well-formatted integer. It's just wrong.

This is the distinction the Limitation page is built around: state interference is not the same as running out of context length. A standard Transformer truncates sharply and visibly at N_max. A linear-attention model degrades silently. That silent failure mode is the single most important take-away of this project.

Other primary sources cited in-app

Yang, S. et al. (2023). Gated Linear Attention Transformers with Hardware-Efficient Training. arXiv:2312.06635.

Sun, Y. et al. (2023). Retentive Network: A Successor to Transformer for Large Language Models. arXiv:2307.08621.

All code is open-source at github.com/yogeshroyal63-beep/Dataforge. See README.md for reproduction steps, AI_DISCLOSURE.md for assistance disclosure, and LICENSES.md for credits.